Department of CSE(AIML), PES University
UE26CS151A -Python for Computational Problem Solving
Laboratory Week 5

Note: Students are required to write the programs along with atleast two outputs for each experiment in the laboratory observation record. The observation record must be signed by the faculty in charge. For every laboratory session, the observation component carries **marks**.

Problem Statements

Q1. Count Occurrences in a Tuple

Write a Python program to create a tuple containing some repeated elements. Count and display the number of times a specified element occurs in the tuple without using the built-in `count()` method.

Q2.Update a Tuple Element

Write a Python program to create a tuple containing the marks of five students. Accept a student position and a new mark from the user. Since tuples are immutable, convert the tuple into a list, update the mark at the specified position, and convert it back into a tuple. Display the original tuple and the updated tuple.

Q3. Find Common Elements in Two Sets

Write a program to find the common elements between two sets.

Q4. Unique Common Elements Using Tuple and Sets

Write a Python program to create two tuples containing the names of students who participated in Event 1 and Event 2. Some students may appear more than once in a tuple. Convert both tuples into sets and display:

1. The unique students who participated in both events.
2. The students who participated only in Event 1.
3. The total number of unique students who participated in either event.

Q5. Student Marks Analysis Using Tuple and Set

Write a Python program to create a tuple containing the names of five students and another tuple containing their corresponding marks. Create a set containing the marks of students who scored above 75. Display the set of qualifying marks and the number of students who scored above 75.

Q6. Student Grade Management Using Tuple and Set

Write a Python program to **accept the names and grades of five students from the user** and store them in two tuples.

1. Create a set containing the **unique grades** obtained by the students.
2. Count the number of students who received grade `'A'`.
3. If no student received grade `'A'`, display an appropriate message.
4. Display the student names, grades, unique grades, and the appropriate result for grade `'A'`.

Q7. Set Relationship Analysis

Write a program to create two sets of integers and determine whether the sets are equal, disjoint, or one is a subset of the other

Q8. Employee Working Hours Analysis using Tuples

Write a Python program to accept the working hours of 7 employees from the user and store them in a tuple. Display the working hours of employees who worked more than 8 hours and count the number of employees who worked more than 8 hours.

Constraint: Do not use the map() function.

Solutions

1. Count Occurrences in a Tuple

```
numbers = (10, 20, 10, 30, 40, 10, 20, 50)

element = 10
count = 0

for value in numbers:
    if value == element:
        count += 1

print("Tuple:", numbers)
print("Element:", element)
print("Number of occurrences:", count)
```

Output
Tuple: (10, 20, 10, 30, 40, 10, 20, 50)
Element: 10
Number of occurrences: 3

2. Update a Tuple Element

```
marks = (78, 85, 92, 67, 88)

print("Original tuple:", marks)

position = int(input("Enter the position to update (1-5): "))
new_mark = int(input("Enter the new mark: "))

marks_list = list(marks)

marks_list[position - 1] = new_mark

marks = tuple(marks_list)

print("Updated tuple:", marks)
```

Output
Original tuple: (78, 85, 92, 67, 88)
Enter the position to update (1-5): 4
Enter the new mark: 75
Updated tuple: (78, 85, 92, 75, 88)

3. Find Common Elements in Two Sets

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A.intersection(B))
```

Output
{3, 4}

4. Unique Common Elements Using Tuple and Sets

```
event1 = ("Anil", "Bhavya", "Charan", "Anil", "Kiran", "Esha")
event2 = ("Bhavya", "Farhan", "Kiran", "Girish", "Bhavya", "Esha")

set1 = set(event1)
set2 = set(event2)

both_events = set1 & set2
only_event1 = set1 - set2
all_students = set1 | set2

print("Students in both events:", both_events)
print("Students only in Event 1:", only_event1)
print("Total unique students:", len(all_students))
```

Output

```
Students in both events: {'Bhavya', 'Kiran', 'Esha'}
Students only in Event 1: {'Anil', 'Charan'}
Total unique students: 7
```

5. Student Marks Analysis Using Tuple and Set

```
students = ("Arun", "Bhavya", "Charan", "Divya", "Kiran")
marks = (82, 68, 91, 75, 88)

qualified_students = set()

for i in range(len(students)):
    if marks[i] > 75:
        qualified_students.add(students[i])

print("Students who scored above 75:", qualified_students)
print("Number of students:", len(qualified_students))
```

Output

```
Students who scored above 75: {'Arun', 'Charan', 'Kiran'}
Number of students: 3
```

Output

```
Average Marks = 84.0
Students scoring above average:
Amit : 85
Riya : 92
Neha : 88
```

6. Student Grade Management Using Tuple and Set

```
students = ()
grades = ()

for i in range(5):
    name = input("Enter student name: ")
    grade = input("Enter grade: ")

    students = students + (name,)
    grades = grades + (grade,)

unique_grades = set(grades)
a_count = 0

for grade in grades:
    if grade == "A":
        a_count += 1

print("\nStudent Names:", students)
print("Grades:", grades)
print("Unique grades:", unique_grades)

if a_count == 0:
    print("No student received grade A")
else:
    print("Number of students who received grade A:", a_count)
```

Output 1

```
Enter student name: Arun
Enter grade: A
Enter student name: Bhavya
Enter grade: B
```

Enter student name: Charan
Enter grade: A
Enter student name: Meera
Enter grade: C
Enter student name: Kiran
Enter grade: B

Student Names: ('Arun', 'Bhavya', 'Charan', 'Meera', 'Kiran')
Grades: ('A', 'B', 'A', 'C', 'B')
Unique grades: {'A', 'B', 'C'}
Number of students who received grade A: 2

Output 2
Enter student name: Arun
Enter grade: B
Enter student name: Bhavya
Enter grade: C
Enter student name: Charan
Enter grade: B
Enter student name: Meera
Enter grade: C
Enter student name: Kiran
Enter grade: B

Student Names: ('Arun', 'Bhavya', 'Charan', 'Meera', 'Kiran')
Grades: ('B', 'C', 'B', 'C', 'B')
Unique grades: {'B', 'C'}
No student received grade A

7. Set Relationship Analysis

```
set1 = {10, 20, 30}
set2 = {10, 20, 30, 40, 50}

if set1 == set2:
    print("The two sets are equal.")
elif set1.isdisjoint(set2):
    print("The two sets are disjoint.")
elif set1.issubset(set2):
    print("Set1 is a subset of Set2.")
elif set2.issubset(set1):
```

```
    print("Set2 is a subset of Set1.")
else:
    print("The sets have some common elements.")
```

Output

Set1 is a subset of Set2.

8. Employee Working Hours Analysis using Tuples

```
working_hours = ()

for i in range(1, 8):
    hours = int(input("Enter working hours for employee " + str(i) + ": "))
    working_hours += (hours,)

count = 0

print("\nEmployees who worked more than 8 hours:")

for i in range(7):
    if working_hours[i] > 8:
        print("Employee", i + 1, ":", working_hours[i], "hours")
        count += 1

print("Number of employees:", count)
```

Output

```
Enter working hours for employee 1: 7
Enter working hours for employee 2: 9
Enter working hours for employee 3: 8
Enter working hours for employee 4: 10
Enter working hours for employee 5: 6
Enter working hours for employee 6: 9
Enter working hours for employee 7: 11
```

Employees who worked more than 8 hours:
Employee 2 : 9 hours
Employee 4 : 10 hours
Employee 6 : 9 hours
Employee 7 : 11 hours
Number of employees: 4

Prepared By,
Divyakeerthi